

Progetto d'esame di Architettura degli Elaboratori

Samuele Caporale - 329096
Matilde Pettinari - 328737

A.A. 2025/2026

Indice

1	Specifica	3
1.1	Scopo del progetto	3
1.2	Specifica algoritmica	3
1.2.1	implementazione in C	3
2	Implementazione in Assembly MIPS64	4
3	Benchmark	5
3.1	Metriche e tipologie	5
4	Tecniche e considerazioni iniziali	6
4.1	Tecniche di ottimizzazione	6
4.2	Considerazioni	6
4.2.1	Accorgimenti di ambiente	6
5	Ottimizzazione Statica	7
5.1	Versione 0	7
5.2	Versione 0 con data forwarding	9
5.3	Versione 1	11
5.4	Versione 2	13
5.5	Versione 3	17
5.6	Versione 4	20
6	Conclusioni Finali	23
6.1	Confronti dati	23

1 Specifica

1.1 Scopo del progetto

Implementare ed ottimizzare l'algoritmo di ordinamento Insertion Sort in Assembly MIPS64.

Le scelte di ottimizzazione verranno fatte con l'ausilio del software WinMIPS64, utilizzando tecniche come: l'Instruction Reordering, Loop Unrolling e Register Renaming.

L'obiettivo finale è quello di migliorare le performance: riducendo i valori delle metriche d'esecuzione e degli stalli.

1.2 Specifica algoritmica

Insertion Sort è un algoritmo di ordinamento iterativo, per ogni i -esima iterazione l'array è considerato suddiviso in due parti:

- una sequenza **già ordinata**: $a[0], \dots, a[i - 1]$ di destinazione.
- una sequenza **da ordinare**: $a[i], \dots, a[n - 1]$ di origine.

L'obiettivo è inserire il valore contenuto in $a[i]$ nella posizione corretta sulla sequenza di destinazione.

Questo viene fatto confrontando l'elemento corrente, con tutti gli elementi della sequenza di destinazione e facendo scorrere il valore. Ogni volta che un elemento viene inserito nella posizione corretta, la sequenza di destinazione cresce di un elemento e la sequenza di origine si riduce di uno.

L'algoritmo termina l'ordinamento quando tutti gli elementi si trovano nella sequenza di destinazione ordinata, ossia $i = n$.

1.2.1 implementazione in C

Listing 1: insertion_sort.c

```
1  /* Insertion Sort */
2  void insertion_sort(int a[], int n) {
3      int i,
4          j,
5          tmp;
6
7      for (i = 1; i < n; i++) {
8          tmp = a[i];
9          for (j = i - 1; j >= 0 && a[j] > tmp; j--) {
10             a[j+1] = a[j];
11         }
12         a[j+1] = tmp;
13     }
14 }
```

2 Implementazione in Assembly MIPS64

Ogni istruzione del codice C deve essere esplicitamente tradotta in istruzioni Assembly.

La dichiarazione degli array avviene esplicitamente nella sezione **.data**, che è dedicata alla memorizzazione di variabili statiche e dati globali, inoltre garantisce che i dati siano presenti in memoria per tutta la durata del programma.

I dati di input (valori interi) vengono memorizzati sulla memoria in locazioni contigue di dimensione pari a 64 bit, l'equivalente di 8 byte.

```
1 .data
2 a: .word 2,4,6,5,0,7,1,3
3 n: .word 8
```

Con la direttiva **.text** inizia la zona di codice da eseguire, le prime istruzioni riguardano il caricamento dei valori e l'indirizzo dell'array dalla memoria ai registri.

L'accesso agli elementi di **a** verrà fatto tramite l'indirizzo base sommato ad un offset di 8 byte.

Il nostro indirizzo base dell'array viene caricato in **r4**, ed in **r2** viene caricata la dimensione dell'array **n**.

Extern loop è composto da:

- **SLT** (Set on Less Than): confronta due registri e imposta il registro di destinazione a 1 se il primo è minore del secondo, altrimenti lo imposta a 0.
- **BEQZ** (Branch if Equal to Zero): verifica se il registro specificato è uguale a zero; in tal caso il controllo del programma salta all'etichetta indicata.
- **DSLL** (Doubleword Shift Left Logical): esegue uno shift logico a sinistra su una doppia parola (64 bit). Viene utilizzata per moltiplicare l'indice dell'array per 8, poiché ogni elemento dell'array occupa 8 byte in memoria.
- **DADD** (Doubleword Add with Overflow): somma due registri a 64 bit gestendo il possibile overflow.
- **LD** (Load Double): carica in un registro una parola di 64 bit dalla memoria.
- **DADDI** (Doubleword Add Immediate with Overflow): aggiunge a un registro un valore immediato esteso a 64 bit, gestendo il possibile overflow.

Annidato troviamo il secondo ciclo for chiamato **inner_loop** il quale si occupa di gestire lo shift degli elementi.

Le istruzioni di salto incondizionato **J** servono a completare il comportamento del ciclo.

Con la direttiva **HALT**, si termina l'esecuzione del programma.

3 Benchmark

3.1 Metriche e tipologie

Per effettuare l'ottimizzazione del codice abbiamo utilizzato il software WinMIPS64 grazie ad esso abbiamo potuto confrontare le diverse versioni del codice Assembly visionando le seguenti statistiche:

Categoria	Tipi
Esecuzione	Cycles, Instructions, CPI
Stalli	RAW, WAW, WAR, Structural, Branch Taken, Branch Misprediction
Dimensione codice	Bytes

Tabella 1: Metriche

- **Cycles:** Il numero dei cicli di clock totali.
- **Instructions:** Il numero totale di istruzioni eseguite.
- **CPI (Clock Cycles per Instruction):** Il rapporto tra cicli ed istruzioni.
- **RAW (Read After Write):** La dipendenza al RAW avviene quando un'istruzione richiede un dato non ancora calcolato.
- **WAR (Write After Read):** Si verifica quando un'istruzione scrive in un registro che è stato letto da un'istruzione precedente, invalidando l'ordine di precedenza, poichè la load fornisce il risultato sui registri a fine della fase di memory access, invece l'istruzione di scrittura direttamente a fine esecuzione.
- **WAW (Write After Write):** La dipendenza al WAW avviene quando più istruzioni scrivono nello stesso registro o locazione di memoria.
- **Structural Stalls:** E' il tentativo di usare la stessa risorsa hardware da parte di diverse istruzioni in modi diversi nello stesso ciclo di clock.
- **Branch Taken Stalls:** Si verifica quando un'istruzione di salto condizionato o incondizionato viene presa, causando l'abort dell'istruzione successiva, con conseguente introduzione di cicli di stallo fino al fetch ed esecuzione dell'istruzione target.
- **Branch Misprediction:** Si verifica quando il processore prevede erroneamente il risultato di un'istruzione di salto (branch) durante l'esecuzione di un programma.
- **Bytes:** La dimensione totale del codice.

4 Tecniche e considerazioni iniziali

4.1 Tecniche di ottimizzazione

Instruction Reordering

Trasformazione che modifica l'ordine di esecuzione delle istruzioni mantenendo invariate le dipendenze di dati e di controllo, con l'obiettivo di minimizzare gli stalli della pipeline e massimizzare l'Instruction-Level Parallelism (ILP).

Loop Unrolling

Tecnica che consiste nel replicare il corpo di un ciclo un numero finito di volte e nel ridurre proporzionalmente il numero di iterazioni, preservando la coerenza logica e riducendo l'overhead di controllo del ciclo.

Register Renaming

Tecnica che assegna registri fisici distinti a variabili logicamente diverse, pur condividendo lo stesso nome architetturale, al fine di eliminare dipendenze di nome (WAR, WAW) senza modificare le dipendenze di tipo RAW.

4.2 Considerazioni

4.2.1 Accorgimenti di ambiente

Per valutare in modo coerente le metriche ottenute sarà necessario resettare la memoria ad ogni test, in modo da valutare l'algoritmo con la stessa configurazione iniziale dei dati, altrimenti nel test successivo, otterremmo le metriche relative all'ordinamento nel caso ottimo, dove la sequenza è già ordinata.

5 Ottimizzazione Statica

5.1 Versione 0

La versione 0 è la prima implementazione eseguita **senza data forwarding**.

Listing 2: v0

```
1  .data
2  a:  .word 2,4,6,5,0,7,1,3
3  n:  .word 8
4
5  .text
6  LD    r2, n(r0)           ; r2 = n = 8
7  DADDI r3, r0, 1           ; r3 = i = 1
8  DADDI r4, r0, a           ; r4 = a (carica l'indirizzo base dell'array in r4)
9
10 extern_loop:
11     SLT  r7, r3, r2        ; r7 = (i < n) ? 1 : 0
12
13     BEQZ r7, end_extern    ; if i >= n, esce dal ciclo
14
15     DSL  r6, r3, 3         ; r6 = i*8 (calcolo offset per 64-bit word)
16     DADD r8, r4, r6        ; r8 = &a[i]
17
18     LD    r9, 0(r8)        ; r9 = tmp = a[i]
19
20     DADDI r5, r3, -1       ; r5 = j = i - 1
21
22 inner_loop:
23     SLT  r10, r5, r0       ; r10 = 1 se j < 0
24
25     BNEZ r10, end_inner    ; se j < 0, esce dal ciclo
26
27     DSL  r13, r5, 3        ; r13 = j * 8 (calcolo offset per 64-bit word)
28     DADD r14, r4, r13      ; r14 = &a[j]
29
30     LD    r11, 0(r14)      ; r11 = a[j]
31
32     SLT  r12, r9, r11      ; r12 = (tmp < a[j]) ? 1 : 0
33
34     BEQZ r12, end_inner    ; if tmp >= a[j], esce dal ciclo
35
36     SD    r11, 8(r14)      ; a[j+1] = a[j]
37
38     DADDI r5, r5, -1       ; j--
39
40     J     inner_loop       ; ripetizione del ciclo interno
41
42 end_inner:
43     DADDI r5, r5, 1        ; r5 = j + 1
44     DSL  r6, r5, 3        ; r6 = j * 8
45
46     DADD r15, r4, r6       ; r15 = &a[j+1]
47
48     SD    r9, 0(r15)       ; r9 = a[j+1] = tmp
49
50     DADDI r3, r3, 1        ; i++
51     J     extern_loop      ; ripetizione del ciclo esterno
52
53 end_extern:
54     HALT
```

Benchmark:

Execution	
Cycles	610
Instructions	274
CPI	2.226
Stalls	
RAW Stalls	303
WAW Stalls	0
WAR Stalls	0
Structural Stalls	0
Branch Taken Stalls	29
Branch Misprediction Stalls	0
Code size	
Bytes	104

Tabella 2: v0

Nella versione 0 sono presenti 303 stalli, si verificano principalmente a causa delle dipendenze di tipo RAW (Read After Write).

Gli stalli possono manifestarsi in diversi contesti, inclusi i casi di dipendenze tra registri (RAW, WAR, WAW) e hazard di controllo dovuti a salti condizionati o incondizionati.

ad esempio il primo stallo RAW si verifica tra le seguenti istruzioni:

Listing 3: Stallo RAW

```
5  .text
6  LD      r2, n(r0)          ; r2 = n = 8
7  → DADDI r3, r0, 1          ; r3 = i = 1
8  DADDI r4, r0, a            ; r4 = a (carica l'indirizzo base dell'array in r4)
9
10 extern_loop:
11 → SLT   r7, r3, r2          ; r7 = (i < n) ? 1 : 0
12
13 BEQZ   r7, end_extern      ; if i >= n, esce dal ciclo
```

*L'istruzione SLT introduce uno stallo, poichè necessita del registro **r3**, il quale non ha ancora terminato la fase di WB, ed essendo in assenza di data forwarding, il registro sarà disponibile solo il ciclo successivo.*

Stallo eliminato nella versione con data forwarding attivo.

5.2 Versione 0 con data forwarding

Questa versione condivide il codice della precedente, ma viene eseguita con il data forwarding abilitato.

Il data forwarding è una tecnica che riduce i conflitti dovuti alle dipendenze dei dati, permettendo di inoltrare i risultati intermedi. In questo modo l'ALU può utilizzare risultati prodotti da istruzioni precedenti senza attendere il completamento dell'intero ciclo di scrittura nel registro, migliorando le prestazioni e riducendo gli stalli della pipeline.

Listing 4: v0

```
1  .data
2  a:  .word 2,4,6,5,0,7,1,3
3  n:  .word 8
4
5  .text
6      LD      r2, n(r0)          ; r2 = n = 8
7      DADDI   r3, r0, 1          ; r3 = i = 1
8      DADDI   r4, r0, a          ; r4 = a (carica l'indirizzo base dell'array in r4)
9
10 extern_loop:
11     SLT     r7, r3, r2          ; r7 = (i < n) ? 1 : 0
12     BEQZ    r7, end_extern      ; if i >= n, esce dal ciclo
13
14     DSL     r6, r3, 3           ; r6 = i*8 (calcolo offset per 64-bit word)
15     DADD    r8, r4, r6          ; r8 = &a[i]
16     LD      r9, 0(r8)          ; r9 = tmp = a[i]
17     DADDI   r5, r3, -1          ; r5 = j = i - 1
18
19 inner_loop:
20     SLT     r10, r5, r0          ; r10 = 1 se j < 0
21     BNEZ    r10, end_inner      ; se j < 0, esce dal ciclo
22
23     DSL     r13, r5, 3           ; r13 = j * 8 (calcolo offset per 64-bit word)
24     DADD    r14, r4, r13        ; r14 = &a[j]
25     LD      r11, 0(r14)         ; r11 = a[j]
26
27     SLT     r12, r9, r11         ; r12 = (tmp < a[j]) ? 1 : 0
28
29
30     BEQZ    r12, end_inner      ; if tmp >= a[j], esce dal ciclo
31     SD      r11, 8(r14)         ; a[j+1] = a[j]
32     DADDI   r5, r5, -1          ; j--
33     J       inner_loop          ; ripetizione del ciclo interno
34
35 end_inner:
36     DADDI   r5, r5, 1           ; r5 = j + 1
37     DSL     r6, r5, 3           ; r6 = j * 8
38     DADD    r15, r4, r6         ; r15 = &a[j+1]
39     SD      r9, 0(r15)          ; r9 = a[j+1] = tmp
40     DADDI   r3, r3, 1           ; i++
41     J       extern_loop        ; ripetizione del ciclo esterno
42
43 end_extern:
44     HALT
```

Benchmark:

Execution	
Cycles	376
Instructions	274
CPI	1.372
Stalls	
RAW Stalls	69
WAW Stalls	0
WAR Stalls	0
Structural Stalls	0
Branch Taken Stalls	29
Branch Misprediction Stalls	0
Code size	
Bytes	104

Tabella 3: v0 con data forwarding

Si può osservare un chiaro miglioramento delle performance, gli stalli RAW sono diminuiti grazie all'attivazione del data forwarding, che ha permesso alle istruzioni ALU utilizzare i valori dei registri senza attendere il ciclo di WB

Conflitti residui:

Listing 5: Stallo RAW

```
10 extern_loop:
11 → SLT   r7, r3, r2      ; r7 = (i < n) ? 1 : 0
12 → BEQZ  r7, end_extern  ; if i >= n, esce dal ciclo
13
14     DSLL  r6, r3, 3      ; r6 = i*8 (calcolo offset per 64-bit word)
15     DADD  r8, r4, r6     ; r8 = &a[i]
16     LD    r9, 0(r8)      ; r9 = tmp = a[i]
17     DADDI r5, r3, -1     ; r5 = j = i - 1
```

L'istruzione BEQZ verifica se il valore in `r7` è zero, in tal caso, esegue un salto all'etichetta `end_extern`. Dato che `r7` è stato aggiornato dalla precedente istruzione SLT, la decisione di salto causa uno stallo di controllo dato che il valore in `r7` non è ancora stato scritto al momento dell'esecuzione di BEQZ. Questo tipo di stallo è dovuto al fatto che deve attendere il risultato della SLT per decidere se eseguire il salto.

5.3 Versione 1

Listing 6: v1

```
1  .data
2  a:  .word 2,4,6,5,0,7,1,3
3  n:  .word 8
4
5  .text
6      LD      r2, n(r0)          ; r2 = n = 8
7      DADDI   r3, r0, 1          ; r3 = i = 1
8      DADDI   r4, r0, a          ; r4 = a (carica l'indirizzo base dell'array in r4)
9
10 extern_loop:
11     SLT     r7, r3, r2          ; r7 = (i < n) ? 1 : 0
12     DSSL    r6, r3, 3          ; r6 = i*8 (calcolo offset per 64-bit word)
13     BEQZ    r7, end_extern      ; if i >= n, esce dal ciclo
14     DADD    r8, r4, r6          ; r8 = &a[i]
15     LD      r9, 0(r8)          ; r9 = tmp = a[i]
16     DADDI   r5, r3, -1          ; r5 = j = i - 1
17
18 inner_loop:
19     SLT     r10, r5, r0         ; r10 = 1 se j < 0
20     DSSL    r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
21     BNEZ    r10, end_inner      ; se j < 0, esce dal ciclo
22     DADD    r14, r4, r13        ; r14 = &a[j]
23     LD      r11, 0(r14)         ; r11 = a[j]
24     SLT     r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
25
26     BEQZ    r12, end_inner      ; if tmp >= a[j], esce dal ciclo
27
28
29     SD      r11, 8(r14)         ; a[j+1] = a[j]
30     DADDI   r5, r5, -1          ; j--
31     J       inner_loop         ; ripetizione del ciclo interno
32
33 end_inner:
34     DADDI   r5, r5, 1          ; r5 = j + 1
35     DSSL    r6, r5, 3          ; r6 = j * 8
36     DADD    r15, r4, r6        ; r15 = &a[j+1]
37     SD      r9, 0(r15)         ; r9 = a[j+1] = tmp
38     DADDI   r3, r3, 1          ; i++
39     J       extern_loop        ; ripetizione del ciclo esterno
40
41 end_extern:
42     HALT
```

Benchmark

Execution	
Cycles	349
Instructions	276
CPI	1.264
Stalls	
RAW Stalls	40
WAW Stalls	0
WAR Stalls	0
Structural Stalls	0
Branch Taken Stalls	29
Branch Misprediction Stalls	0
Code size	
Bytes	104

Tabella 4: v1

Tecnica di ottimizzazione applicata

- Instruction Reordering

File sorgente: v1.s

Conflitti risolti:

Risolto lo stallo ad inizio ciclo spostando l'istruzione DSLR in modo da mascherare lo stallo RAW tra registri precedentemente mostrato.

Listing 7: Instruction reordering

```

10 extern_loop:
11     SLT    r7, r3, r2        ; r7 = (i < n) ? 1 : 0
12 → DSLR    r6, r3, 3         ; r6 = i*8 (calcolo offset per 64-bit word)
13     BEQZ   r7, end_extern    ; if i >= n, esce dal ciclo
14
15 → DSLR r6, r3, 3
16     DADD   r8, r4, r6        ; r8 = &a[i]
17     LD     r9, 0(r8)         ; r9 = tmp = a[i]
18
19     DADDI   r5, r3, -1        ; r5 = j = i - 1

```

Allo stesso modo viene applicata la tecnica di instruction reordering nella seguente sequenza di istruzioni.

Listing 8: Instruction reordering

```

22 inner_loop:
23     SLT    r10, r5, r0        ; r10 = 1 se j < 0
24 → DSLR    r13, r5, 3         ; r13 = j * 8 (calcolo offset per 64-bit word)
25     BNEZ   r10, end_inner    ; se j < 0, esce dal ciclo
26
27 → DSLR r13, r5, 3
28     DADD   r14, r4, r13      ; r14 = &a[j]
29     LD     r11, 0(r14)       ; r11 = a[j]
30     SLT    r12, r9, r11      ; r12 = (tmp < a[j]) ? 1 : 0

```

Conflitti residui

Rimane ancora da risolvere il conflitto tra LD e SLT (Data Hazard)

5.4 Versione 2

Listing 9: v2

```
1  .data
2  a:  .word 2,4,6,5,0,7,1,3
3  n:  .word 8
4
5  .text
6      LD      r2, n(r0)          ; r2 = n = 8
7      DADDI   r3, r0, 1          ; r3 = i = 1
8      DADDI   r4, r0, a          ; r4 = a (carica l'indirizzo base dell'array in r4)
9
10 extern_loop:
11     SLT     r7, r3, r2          ; r7 = (i < n) ? 1 : 0
12     DSSL    r6, r3, 3          ; r6 = i*8 (calcolo offset per 64-bit word)
13     BEQZ    r7, end_extern      ; if i >= n, esce dal ciclo
14     DADD    r8, r4, r6          ; r8 = &a[i]
15     LD      r9, 0(r8)          ; r9 = tmp = a[i]
16     DADDI   r5, r3, -1          ; r5 = j = i - 1
17
18 inner_loop:
19     SLT     r10, r5, r0         ; r10 = 1 se j < 0
20     DSSL    r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
21     DADD    r14, r4, r13        ; r14 = &a[j]
22     LD      r11, 0(r14)         ; r11 = a[j]
23     BNEZ    r10, end_inner      ; se j < 0, esce dal ciclo
24     SLT     r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
25
26     BEQZ    r12, end_inner      ; if tmp >= a[j], esce dal ciclo
27     SD      r11, 8(r14)         ; a[j+1] = a[j]
28     DADDI   r5, r5, -1          ; j--
29     ; ----- Seconda parte -----
30     SLT     r10, r5, r0         ; r10 = 1 se j < 0
31     DSSL    r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
32     DADD    r14, r4, r13        ; r14 = &a[j]
33     LD      r11, 0(r14)         ; r11 = a[j]
34     BNEZ    r10, end_inner      ; se j < 0, esce dal ciclo
35     SLT     r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
36
37     BEQZ    r12, end_inner      ; if tmp >= a[j], esce dal ciclo
38     SD      r11, 8(r14)         ; a[j+1] = a[j]
39     DADDI   r5, r5, -1          ; j--
40
41     J       inner_loop          ; ripetizione del ciclo interno
42
43 end_inner:
44     DADDI   r5, r5, 1           ; r5 = j + 1
45     DSSL    r6, r5, 3           ; r6 = j * 8
46     DADD    r15, r4, r6         ; r15 = &a[j+1]
47     SD      r9, 0(r15)          ; r9 = a[j+1] = tmp
48     DADDI   r3, r3, 1           ; i++
49     J       extern_loop         ; ripetizione del ciclo esterno
50
51 end_extern:
52     HALT
```

Benchmark:

In linea con le osservazioni precedenti si è andato a ridurre il numero di stalli RAW e Branch Taken Stalls, segue la riduzione del numero di cicli totali, istruzioni e CPI complessivo sempre più vicino al valore ideale di 1, l'unica metrica ad essere peggiorata è naturalmente la dimensione del codice.

Execution	
Cycles	315
Instructions	270
CPI	1.167
Stalls	
RAW Stalls	20
WAW Stalls	0
WAR Stalls	0
Structural Stalls	0
Branch Taken Stalls	21
Branch Misprediction Stalls	0
Code size	
Bytes	140

Tabella 5: v2

Tecnica di ottimizzazione applicata

- Loop Unrolling
- Register Reordering

File sorgente: v2.s

Conflitti da risolvere:

Qui si possono osservare le istruzioni che generano stalli di tipo RAW

Listing 10: Stalli RAW

```

22 inner_loop:
23 ...
24 → LD      r11, 0(r14)      ; r11 = a[j]
25 → SLT     r12, r9, r11     ; r12 = (tmp < a[j]) ? 1 : 0
26 → BEQZ    r12, end_inner   ; if tmp >= a[j], esce dal ciclo
27 ...

```

Per iniziare applichiamo il Loop Unrolling.

L'applicazione del Loop Unrolling introduce in generale vincoli sull'input, i quali impongono di rispettare il fattore di unrolling; tuttavia, grazie alla guardia $j < 0$ ripetuta, è possibile gestire correttamente qualsiasi numero di elementi, garantendo l'uscita dal ciclo anche durante l'esecuzione della seconda parte del corpo srotolato.

Listing 11: Loop Unrolling

```

22 inner_loop:
23     SLT r10, r5, r0          ; r10 = 1 se j < 0
24
25     DSLL r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
26     BNEZ r10, end_inner      ; se j < 0, esce dal ciclo
27 → DADD r14, r4, r13          ; r14 = &a[j]
28 → LD r11, 0(r14)             ; r11 = a[j]
29 → SLT r12, r9, r11           ; r12 = (tmp < a[j]) ? 1 : 0
30 → BEQZ r12, end_inner        ; if tmp >= a[j], esce dal ciclo
31
32     SD r11, 8(r14)           ; a[j+1] = a[j]
33     DADDI r5, r5, -1         ; j--
34     ; ----- Seconda parte -----
35     SLT r10, r5, r0          ; r10 = 1 se j < 0
36
37     DSLL r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
38
39     BNEZ r10, end_inner      ; se j < 0, esce dal ciclo
40 → DADD r14, r4, r13          ; r14 = &a[j]
41 → LD r11, 0(r14)             ; r11 = a[j]
42 → SLT r12, r9, r11           ; r12 = (tmp < a[j]) ? 1 : 0
43 → BEQZ r12, end_inner        ; if tmp >= a[j], esce dal ciclo
44
45     SD r11, 8(r14)           ; a[j+1] = a[j]
46     DADDI r5, r5, -1         ; j--
47
48     J inner_loop             ; ripetizione del ciclo interno

```

Per risolvere il primo stallo RAW tra l'istruzione *LD* e *SLT*, possiamo inserire tra le due istruzioni operazioni che usano registri non coinvolti nelle altre istruzioni del loop.

L'idea principale è spostare le istruzioni *LD* e *DADD* sopra la *BNEZ*, in modo da distanziare *LD* da *SLT*.

In questo modo si riesce a mascherare il primo stallo RAW. Va considerato che, nel caso in cui la condizione per il branch *BEQZ* sia soddisfatta, si eseguirebbero comunque le istruzioni *LD* e *DADD* non necessarie (poichè anteposte), ma l'impatto dipende dalla distribuzione dei valori nell'array.

Listing 12: Register Reordering

```

22 inner_loop:
23     SLT r10, r5, r0          ; r10 = 1 se j < 0
24     DSLL r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
25
26 → DADD r14, r4, r13          ; r14 = &a[j]
27 → LD r11, 0(r14)             ; r11 = a[j]
28
29 → BNEZ r10, end_inner        ; se j < 0, esce dal ciclo
30 → SLT r12, r9, r11           ; r12 = (tmp < a[j]) ? 1 : 0
31 → BEQZ r12, end_inner        ; if tmp >= a[j], esce dal ciclo
32     SD r11, 8(r14)           ; a[j+1] = a[j]
33     DADDI r5, r5, -1         ; j--
34     ; ----- Secondo Loop -----
35     SLT r10, r5, r0          ; r10 = 1 se j < 0
36     DSLL r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
37
38 → DADD r14, r4, r13          ; r14 = &a[j]
39 → LD r11, 0(r14)             ; r11 = a[j]
40
41 → BNEZ r10, end_inner        ; se j < 0, esce dal ciclo
42 → SLT r12, r9, r11           ; r12 = (tmp < a[j]) ? 1 : 0
43 → BEQZ r12, end_inner        ; if tmp >= a[j], esce dal ciclo
44     SD r11, 8(r14)           ; a[j+1] = a[j]
45     DADDI r5, r5, -1         ; j--
46
47     J inner_loop             ; ripetizione del ciclo interno

```

Conflitti risolti:

- **Branch Taken Stall:**

Grazie al Loop Unrolling si è andati ad intervenire sul numero di Branch Taken Stall, poichè replicando il corpo dell'inner_loop il numero di iterazioni necessarie al ciclo interno sarà ridotto, e di conseguenza minor numero di Branch Taken Stall dovuti all'abort relativo al salto incondizionato finale J inner_loop.

- **RAW:**

Con il Register Reordering siamo andati a mascherare uno stallo, spostando le istruzioni di calcolo indirizzo base e caricamento dell'operando, prima del controllo di guardia del ciclo, separando quindi LD da SLT.

da notare che grazie al data forwarding questo scambio non provoca ulteriori stalli dovuti all'uso del registro r13 poichè disponibile subito dopo la fase di EX .

Conflitti residui

Rimane ancora da risolvere il conflitto RAW tra SLT e BEQZ per entrambe le parti del corpo. Poichè la SLT rende disponibile r12 alla fine della fase di EX grazie al data forwarding, ma la BEQZ controlla la condizione di salto già in fase di ID, da cui si introduce 1 stallo RAW

Listing 13: Stalli RAW

```
22 inner_loop:
23     SLT r10, r5, r0      ; r10 = 1 se j < 0
24     DSLL r13, r5, 3      ; r13 = j * 8 (calcolo offset per 64-bit word)
25
26     DADD r14, r4, r13     ; r14 = &a[j]
27     LD r11, 0(r14)        ; r11 = a[j]
28
29     BNEZ r10, end_inner   ; se j < 0, esce dal ciclo
30 → SLT r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
31 → BEQZ r12, end_inner     ; if tmp >= a[j], esce dal ciclo
32     SD r11, 8(r14)        ; a[j+1] = a[j]
33     DADDI r5, r5, -1      ; j--
34                               ; ----- Secondo Loop -----
35     SLT r10, r5, r0      ; r10 = 1 se j < 0
36     DSLL r13, r5, 3      ; r13 = j * 8 (calcolo offset per 64-bit word)
37
38     DADD r14, r4, r13     ; r14 = &a[j]
39     LD r11, 0(r14)        ; r11 = a[j]
40
41     BNEZ r10, end_inner   ; se j < 0, esce dal ciclo
42 → SLT r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
43 → BEQZ r12, end_inner     ; if tmp >= a[j], esce dal ciclo
44     SD r11, 8(r14)        ; a[j+1] = a[j]
45     DADDI r5, r5, -1      ; j--
46
47     J inner_loop          ; ripetizione del ciclo interno
```


5.5 Versione 3

Listing 14: v3

```
1  .data
2  a:  .word 2,4,6,5,0,7,1,3
3  n:  .word 8
4
5  .text
6      LD      r2, n(r0)          ; r2 = n = 8
7      DADDI   r3, r0, 1          ; r3 = i = 1
8      DADDI   r4, r0, a          ; r4 = a (carica l'indirizzo base dell'array in r4)
9
10 extern_loop:
11     SLT     r7, r3, r2          ; r7 = (i < n) ? 1 : 0
12     DSSL    r6, r3, 3           ; r6 = i*8 (calcolo offset per 64-bit word)
13     BEQZ    r7, end_extern      ; if i >= n, esce dal ciclo
14     DADD    r8, r4, r6          ; r8 = &a[i]
15     LD      r9, 0(r8)          ; r9 = tmp = a[i]
16     DADDI   r5, r3, -1         ; r5 = j = i - 1
17
18 inner_loop:
19     DSSL    r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
20     DADD    r14, r4, r13        ; r14 = &a[j]
21     LD      r11, 0(r14)         ; r11 = a[j]
22     SLT     r10, r5, r0         ; r10 = 1 se j < 0
23     SLT     r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
24     BNEZ    r10, end_inner      ; se j < 0, esce dal ciclo
25     BEQZ    r12, end_inner      ; if tmp >= a[j], esce dal ciclo
26     SD      r11, 8(r14)         ; a[j+1] = a[j]
27     DADDI   r5, r5, -1         ; j--
28     ; ----- Seconda parte -----
29     DSSL    r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
30     DADD    r14, r4, r13        ; r14 = &a[j]
31     LD      r11, 0(r14)         ; r11 = a[j]
32     SLT     r10, r5, r0         ; r10 = 1 se j < 0
33     SLT     r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
34     BNEZ    r10, end_inner      ; se j < 0, esce dal ciclo
35     BEQZ    r12, end_inner      ; if tmp >= a[j], esce dal ciclo
36     SD      r11, 8(r14)         ; a[j+1] = a[j]
37     DADDI   r5, r5, -1         ; j--
38     J       inner_loop         ; ripetizione del ciclo interno
39
40 end_inner:
41     DADDI   r5, r5, 1          ; r5 = j + 1
42     DSSL    r6, r5, 3           ; r6 = j * 8
43     DADD    r15, r4, r6         ; r15 = &a[j+1]
44     SD      r9, 0(r15)         ; r9 = a[j+1] = tmp
45     DADDI   r3, r3, 1          ; i++
46     J       extern_loop        ; ripetizione del ciclo esterno
47
48 end_extern:
49     HALT
```

Benchmark:

Le performance sono complessivamente migliorate, il numero di Stalli RAW sono stati ridotti a zero.

Execution	
Cycles	296
Instructions	271
CPI	1.092
Stalls	
RAW Stalls	0
WAW Stalls	0
WAR Stalls	0
Structural Stalls	0
Branch Taken Stalls	21
Branch Misprediction Stalls	0
Code size	
Bytes	140

Tabella 6: v3

Tecnica di ottimizzazione applicata

- Register Reordering

File sorgente: v3.s

Conflitti da risolvere:

Il conflitto precedentemente illustrato si può osservare nel seguente codice:

Listing 15: Stalli RAW

```

18 inner_loop:
19     SLT r10, r5, r0      ; r10 = 1 se j < 0
20     DSLL r13, r5, 3      ; r13 = j * 8 (calcolo offset per 64-bit word)
21     DADD r14, r4, r13    ; r14 = &a[j]
22     LD r11, 0(r14)       ; r11 = a[j]
23     BNEZ r10, end_inner  ; se j < 0, esce dal ciclo
24 →  SLT r12, r9, r11      ; r12 = (tmp < a[j]) ? 1 : 0
25
26 →  BEQZ r12, end_inner   ; if tmp >= a[j], esce dal ciclo
27     SD r11, 8(r14)       ; a[j+1] = a[j]
28     DADDI r5, r5, -1     ; j--
29     ; ----- Seconda parte -----
30     SLT r10, r5, r0      ; r10 = 1 se j < 0
31     DSLL r13, r5, 3      ; r13 = j * 8 (calcolo offset per 64-bit word)
32     DADD r14, r4, r13    ; r14 = &a[j]
33     LD r11, 0(r14)       ; r11 = a[j]
34     BNEZ r10, end_inner  ; se j < 0, esce dal ciclo
35 →  SLT r12, r9, r11      ; r12 = (tmp < a[j]) ? 1 : 0
36
37 →  BEQZ r12, end_inner   ; if tmp >= a[j], esce dal ciclo
38     SD r11, 8(r14)       ; a[j+1] = a[j]
39     DADDI r5, r5, -1     ; j--
40
41     J inner_loop         ; ripetizione del ciclo interno

```

L'idea è quella di spostare *BNEZ* un'istruzione dopo, tra la *LD* e la *BEQZ*.

Listing 16: Register Reordering

```

20 ...
21 DADD r14, r4, r13 ; r14 = &a[j]
22 LD r11, 0(r14) ; r11 = a[j]
23 → BNEZ r10, end_inner
24 SLT r12, r9, r11 ; r12 = (tmp < a[j]) ? 1 : 0
25 → BNEZ r10, end_inner ; se j < 0, esce dal ciclo
26 BEQZ r12, end_inner ; if tmp >= a[j], esce dal ciclo
27 ...

```

Questa ottimizzazione ci porta a rimuovere lo stallo desiderato, ma con la conseguenza di un nuovo stallo RAW tra la LD e la SLT.

Si applicherà nuovamente l'Instruction reordering spostando l'istruzione iniziale SLT r10, r5, r0 tra le due che generano il nuovo conflitto.

Conflitti risolti:

- Stalli RAW

Con questa sequenza di ottimizzazioni si è riusciti a ridurre a 0 il numero di stalli RAW.

Listing 17: Stalli RAW

```

18 inner_loop:
19 DSLL r13, r5, 3 ; r13 = j * 8 (calcolo offset per 64-bit word)
20 DADD r14, r4, r13 ; r14 = &a[j]
21 LD r11, 0(r14) ; r11 = a[j]
22 SLT r10, r5, r0 ; r10 = 1 se j < 0
23 SLT r12, r9, r11 ; r12 = (tmp < a[j]) ? 1 : 0
24 BNEZ r10, end_inner ; se j < 0, esce dal ciclo
25 BEQZ r12, end_inner ; if tmp >= a[j], esce dal ciclo
26 SD r11, 8(r14) ; a[j+1] = a[j]
27 DADDI r5, r5, -1 ; j--
28 ; ----- Seconda parte -----
29 DSLL r13, r5, 3 ; r13 = j * 8 (calcolo offset per 64-bit word)
30 DADD r14, r4, r13 ; r14 = &a[j]
31 LD r11, 0(r14) ; r11 = a[j]
32 SLT r10, r5, r0 ; r10 = 1 se j < 0
33 SLT r12, r9, r11 ; r12 = (tmp < a[j]) ? 1 : 0
34 BNEZ r10, end_inner ; se j < 0, esce dal ciclo
35 BEQZ r12, end_inner ; if tmp >= a[j], esce dal ciclo
36 SD r11, 8(r14) ; a[j+1] = a[j]
37 DADDI r5, r5, -1 ; j--
38 J inner_loop ; ripetizione del ciclo interno

```

Conflitti residui

Rimangono da risolvere i conflitti di tipo Branch Taken Stalls, questi come detto in precedenza sono dovuti a salti condizionati e incondizionati, risolvibili con un'ulteriore applicazione di Loop Unrolling.

5.6 Versione 4

```

18 .data
19 a: .word 2,4,6,5,0,7,1,3
20 n: .word 8
21
22 .text
23 LD      r2, n(r0)           ; r2 = n = 8
24 DADDI r3, r0, 1             ; r3 = i = 1
25 DADDI r4, r0, a             ; r4 = a (carica l'indirizzo base dell'array in r4)
26
27 extern_loop:
28     SLT   r7, r3, r2         ; r7 = (i < n) ? 1 : 0
29     DSLL  r6, r3, 3          ; r6 = i*8 (calcolo offset per 64-bit word)
30     BEQZ  r7, end_extern     ; if i >= n, esce dal ciclo
31     DADD  r8, r4, r6          ; r8 = &a[i]
32     LD    r9, 0(r8)          ; r9 = tmp = a[i]
33     DADDI r5, r3, -1         ; r5 = j = i - 1
34
35 inner_loop:
36     DSLL  r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
37     DADD  r14, r4, r13        ; r14 = &a[j]
38     LD    r11, 0(r14)         ; r11 = a[j]
39     SLT   r10, r5, r0         ; r10 = 1 se j < 0
40     SLT   r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
41     BNEZ  r10, end_inner      ; se j < 0, esce dal ciclo
42     BEQZ  r12, end_inner      ; if tmp >= a[j], esce dal ciclo
43     SD    r11, 8(r14)         ; a[j+1] = a[j]
44     DADDI r5, r5, -1          ; j--
45     ; ----- Seconda parte -----
46     DSLL  r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
47     DADD  r14, r4, r13        ; r14 = &a[j]
48     LD    r11, 0(r14)         ; r11 = a[j]
49     SLT   r10, r5, r0         ; r10 = 1 se j < 0
50     SLT   r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
51     BNEZ  r10, end_inner      ; se j < 0, esce dal ciclo
52     BEQZ  r12, end_inner      ; if tmp >= a[j], esce dal ciclo
53     SD    r11, 8(r14)         ; a[j+1] = a[j]
54     DADDI r5, r5, -1          ; j--
55     ; ----- Terza parte -----
56     DSLL  r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
57     DADD  r14, r4, r13        ; r14 = &a[j]
58     LD    r11, 0(r14)         ; r11 = a[j]
59     SLT   r10, r5, r0         ; r10 = 1 se j < 0
60     SLT   r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
61     BNEZ  r10, end_inner      ; se j < 0, esce dal ciclo
62     BEQZ  r12, end_inner      ; if tmp >= a[j], esce dal ciclo
63     SD    r11, 8(r14)         ; a[j+1] = a[j]
64     DADDI r5, r5, -1          ; j--
65     ; ----- Quarta parte -----
66     DSLL  r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
67     DADD  r14, r4, r13        ; r14 = &a[j]
68     LD    r11, 0(r14)         ; r11 = a[j]
69     SLT   r10, r5, r0         ; r10 = 1 se j < 0
70     SLT   r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
71     BNEZ  r10, end_inner      ; se j < 0, esce dal ciclo
72     BEQZ  r12, end_inner      ; if tmp >= a[j], esce dal ciclo
73     SD    r11, 8(r14)         ; a[j+1] = a[j]
74     DADDI r5, r5, -1          ; j--
75     J     inner_loop          ; ripetizione del ciclo interno
76
77 end_inner:
78     DADDI r5, r5, 1           ; r5 = j + 1
79     DSLL  r6, r5, 3          ; r6 = j * 8
80     DADD  r15, r4, r6         ; r15 = &a[j+1]
81     SD    r9, 0(r15)          ; r9 = a[j+1] = tmp
82     DADDI r3, r3, 1           ; i++
83     J     extern_loop        ; ripetizione del ciclo esterno
84
85 end_extern:
86     HALT

```

Benchmark:

Metriche relative ai dati di input misti iniziali.

Si ha un miglioramento su tutte le metriche a discapito della Code size.

Execution	
Cycles	290
Instructions	268
CPI	1.082
Stalls	
RAW Stalls	0
WAW Stalls	0
WAR Stalls	0
Structural Stalls	0
Branch Taken Stalls	18
Branch Misprediction Stalls	0
Code size	
Bytes	212

Tabella 7: v4

Tecnica di ottimizzazione applicata

- Loop Unrolling

File sorgente: v4.s

Conflitti da risolvere:

Rimangono solamente i 21 Branch Taken Stalls; a meno di un loop unrolling totale, questo numero sarà sempre maggiore di 0, poiché sono stalli imprescindibili dovuti al fatto che, dopo un salto, la Control Unit invalida l'istruzione successiva con l'ABORT e la pipeline effettua il fetch della nuova istruzione target.

Analisi dei cicli:

Prima di applicare il loop unrolling, è necessario valutare quale ciclo convenga srotolare.

- `extern_loop`
Il ciclo esterno dipende dal numero di elementi dell'array n . Il numero di stalli generato è lineare in n , per n piccoli il numero di Branch Taken Stalls è trascurabile. Inoltre, srotolare il ciclo esterno obbligherebbe a vincolare il numero di elementi in input.
- `inner_loop`
Al contrario, il ciclo interno esegue mediamente un numero di iterazioni che varia da 1 (caso ottimo, array in ordine crescente) a $n - 1$ (caso pessimo, array in ordine decrescente), e questo ciclo viene ripetuto $n - 1$ volte per ogni iterazione del ciclo esterno:

$$\frac{n(n-1)}{2}$$

Di conseguenza, il numero totale di Branch Taken Stall generati dal ciclo interno è lineare nel caso ottimo rispetto al numero di elementi, mentre diventa quadratico nel caso pessimo.

Per questi motivi, conviene applicare un **unrolling parziale del ciclo interno**, riducendo mediamente (in base alla distribuzione dei valori di **a**) gli stalli quadratici, mentre il ciclo esterno può rimanere invariato, preservando la flessibilità rispetto alla dimensione dell'array.

Conflitti risolti:

- riduzione dei Branch Taken Stalls

Listing 18: Loop Unrolling

```

22
23 inner_loop:
24     DSLL  r13, r5, 3          ; r13 = j * 8 (calcolo offset per 64-bit word)
25     DADD  r14, r4, r13        ; r14 = &a[j]
26     LD    r11, 0(r14)         ; r11 = a[j]
27     SLT   r10, r5, r0         ; r10 = 1 se j < 0
28     SLT   r12, r9, r11        ; r12 = (tmp < a[j]) ? 1 : 0
29     BNEZ  r10, end_inner      ; se j < 0, esce dal ciclo
30     BEQZ  r12, end_inner      ; if tmp >= a[j], esce dal ciclo
31     SD    r11, 8(r14)         ; a[j+1] = a[j]
32     DADDI r5, r5, -1          ; j--
33     ...                       ; ----- Seconda parte -----
34     ...                       ; ----- Terza parte -----
35     ...                       ; ----- Quarta parte -----
36     J     inner_loop          ; ripetizione del ciclo interno

```

Conflitti residui

I conflitti di tipo Branch Taken Stalls sono diminuiti coerentemente con il nostro ragionamento, ma grazie a questa ottimizzazione siamo andati a migliorare le performance soprattutto per il caso pessimo, infatti dai test del caso ottimo e pessimo si può notare che il numero di stalli rimane molto simile (15 ottimo 19 pessimo). a differenza della versione precedente dove nel caso pessimo si aveva un numero molto più grande (15 ottimo e 27 pessimo) di Branch taken stall.

Execution Comparison			
	Caso Ottimo	Caso Pessimo	Input Fissato
Cycles	159	412	290
Instructions	140	389	268
CPI	1.136	1.059	1.082
RAW Stalls	0	0	0
WAW Stalls	0	0	0
WAR Stalls	0	0	0
Structural Stalls	0	0	0
Branch Taken Stalls	15	19	18
Branch Misprediction Stalls	0	0	0
Code size (Bytes)	212	212	212

Tabella 8: Confronto tra caso ottimo, caso pessimo e input fissato.

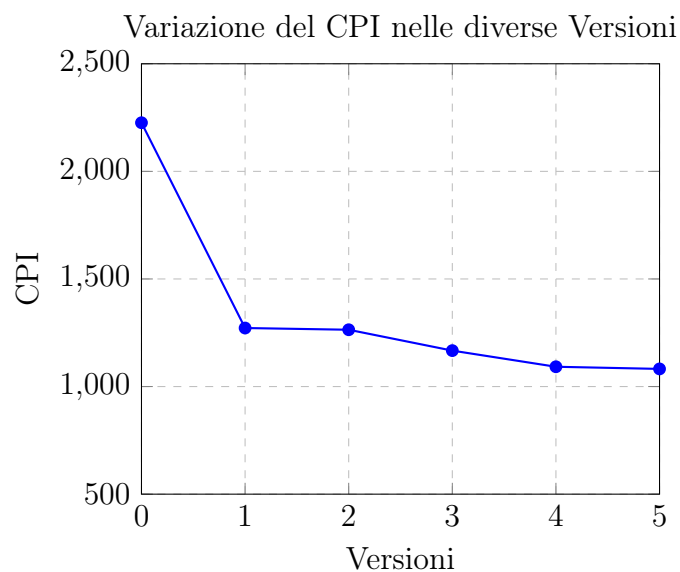
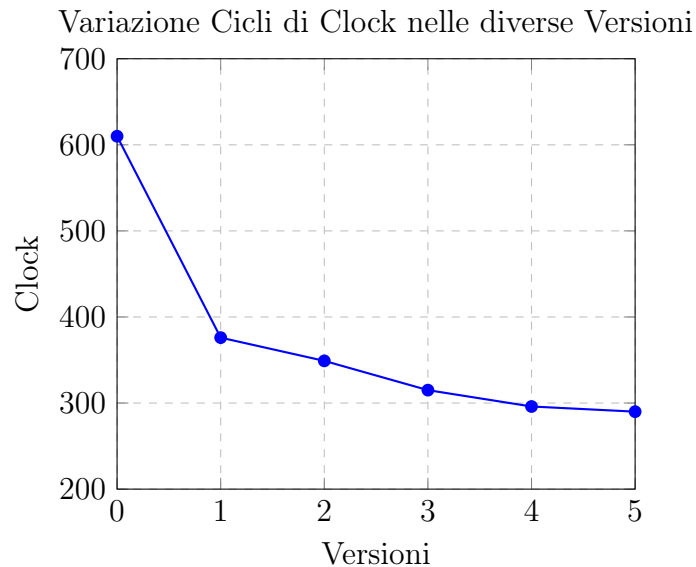
6 Conclusioni Finali

6.1 Confronti dati

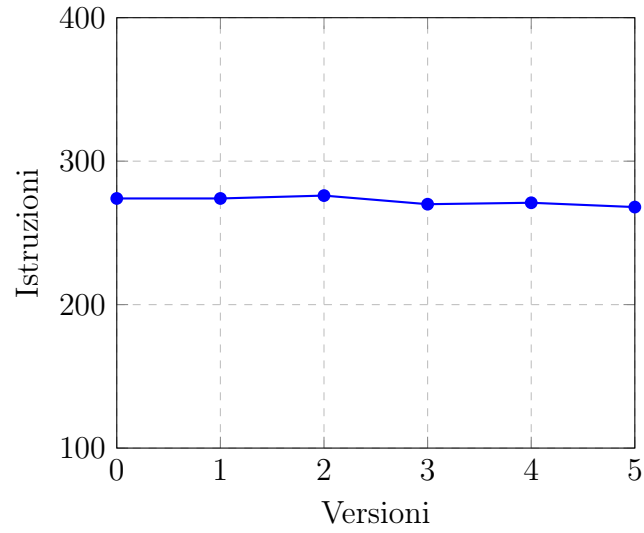
Analizzando le varie versioni del codice di ordinamento, si osserva un miglioramento sostanziale nelle prestazioni grazie al riordino delle istruzioni e all'unrolling del ciclo interno. Queste due tecniche hanno contribuito a ridurre il numero di cicli e il CPI, migliorando l'efficienza globale del codice. Il riordino delle istruzioni ha permesso di ridurre i RAW stalls, mentre l'unrolling ha minimizzato il numero di salti condizionati, rendendo l'esecuzione più fluida ma andando a raddoppiare la dimensione del codice.

Il miglioramento principale è stato ottenuto combinando il Register Reordering e il Loop Unrolling, il che ha portato a una riduzione consistente sia nel numero di cicli che nel numero di stalli.

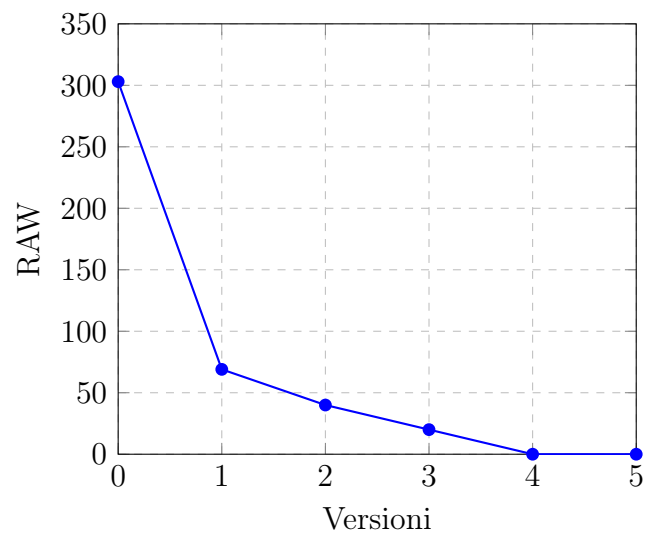
I seguenti grafici illustrano visivamente l'evoluzione delle prestazioni attraverso le varie versioni del codice:



Variazione delle istruzioni nelle diverse Versioni



Variazione degli stalli RAW nelle diverse Versioni



Variazione degli stalli BTS nelle diverse Versioni

